FOUNDER BRIEF

FaceByYou — Infrastructure Overview

How the backend & cloud infrastructure for our AI beauty + skincare MVP is built, what it costs, and how it stays secure.

AWS Account: **Face by You** (708761843741) • Region: us-east-1
Environment: **dev** • Prepared for the founding team

1. The 30-Second Summary

We have a working, secure cloud backend running on Amazon Web Services (AWS). It can accept face photos/videos from the mobile app, run AI analysis, return skincare & makeup recommendations, and store user history — all behind login. Code ships automatically to the server when engineers merge to the main branch. Everything is defined as code, so we can recreate or scale it on demand.

1
cloud account, fully under our control

~$15
/month current running cost (dev)

17+
infrastructure pieces, all automated

0
secrets or passwords stored in code

2. What "Infrastructure" Means Here

Think of it as the digital factory behind the app. The phone app is the storefront; the infrastructure is the warehouse, machinery, security guards, and delivery trucks that make it actually work. Below is every part and why it exists.

Piece	AWS Service	Why it matters (plain English)
The server	EC2	The always-on computer that runs our backend code and answers the app's requests.
Photo/video storage	S3	Where user face images & videos are stored — private, encrypted, auto-deleted after 30 days in dev.
Database	DynamoDB	Stores users, scan results, and history. Scales automatically; we pay only for what we use.
Secret vault	Secrets Manager	Keeps API keys & passwords locked away — never in the code or on anyone's laptop.
Logs & monitoring	CloudWatch	Records what happens so we can see errors, crashes, and usage.
Permissions	IAM	Controls exactly what each part is allowed to do — least privilege, like giving keys only to the rooms someone needs.

Auto-deploy pipeline	GitHub Actions + SSM	When engineers push code, it tests and ships to the server automatically — no manual logins.

3. How a User Request Flows

This is what happens, end to end, when someone scans their face in the app:

- **1. Capture & upload** — The app records a short video and uploads it *directly* to secure storage (S3) using a temporary, single-use link. Large media never passes through our server, which keeps it fast and cheap.
- **2. Analyze** — The app asks the backend to analyze the upload. The AI analysis service returns skin metrics, a skin-age estimate, and makeup recommendations (with a clear medical disclaimer).
- **3. Store & return** — Results are saved to the database under that user and sent back to the app, building their progress history over time.
- **4. Login on everything** — Every request requires a valid login token, so one user can never see another's data.

4. Security & Privacy (Investor- & Apple-Ready)

Because we handle **face data**, security was built in from day one, not bolted on:

> - Storage buckets are **fully private** and **encrypted** — no public access possible.
> - Upload links are **temporary** (expire in 15 minutes) and size-limited.
> - The server uses **least-privilege permissions** — it can only touch the exact resources it needs.
> - All API keys live in a **secret vault**, never in code or chat.
> - Deployments use **short-lived, key-less credentials** — there are no permanent passwords for an attacker to steal.

> **Action items before public launch:** (1) replace the temporary admin access key with a proper limited user, (2) add HTTPS/TLS encryption on the public address, (3) publish a privacy policy, terms, and face-data usage disclosure, and (4) lock down remote-admin access to our office/VPN only.

5. What It Costs

Item	Dev (now)	Notes
Server (EC2 t3.small)	~$15/mo	The only fixed cost. Can be paused when not in use.
Storage (S3)	cents	Pennies per GB; old uploads auto-expire.
Database (DynamoDB)	cents	Pay-per-request; near zero at low volume.
Secrets, logs, pipeline	< $2/mo	Minimal at MVP scale.
Total (dev)	**~$15–20/mo**	Scales with users; designed to grow gradually, not spike.

6. Environments & Growth Path

We use a standard three-stage setup so we never test on real users:

Environment	Purpose	Status

Dev	Engineers build and test freely	Live now
Staging	Final pre-launch testing	Ready to provision (same code, one command)
Production	Real users	Provision when ready for launch

Scaling later is straightforward: add a load balancer + auto-scaling so the app can handle thousands of users and deploy with zero downtime, and move heavy AI models to dedicated GPU services. The foundation we built supports this without a rewrite.

7. What the Founder Should Know / Decide

- We **own** the entire stack in our AWS account — no vendor lock-in beyond AWS itself.
- The MVP runs on a **single server** today (one point of failure) — fine for testing, upgrade before heavy marketing.
- The **AI analysis is a working placeholder** that returns structured results; plugging in the real computer-vision models is the next major engineering milestone.
- Decisions needed soon: login provider (AWS Cognito vs Firebase), and timing for HTTPS + privacy/legal docs.